

Refuerzo de arreglos y representación en memoria

Arreglo unidimensional

Un arreglo unidimensional permite almacenar varios valores del mismo tipo usando un solo identificador.

Ejemplo:

```
int numeros[5];
```

Esto reserva espacio para cinco enteros consecutivos en memoria.

Cada posición se identifica mediante un índice:

```
numeros[0]  
numeros[1]  
numeros[2]  
numeros[3]  
numeros[4]
```

Ejemplo de carga e impresión

```
for(int i = 0; i < 5; i++) {  
    scanf("%d", &numeros[i]);  
}  
  
for(int i = 0; i < 5; i++) {  
    printf("%d ", numeros[i]);  
}
```

Representación en memoria

Si el arreglo contiene:

```
10 20 30 40 50
```

En memoria puede representarse así:

```
0x1000 → 10  
0x1004 → 20  
0x1008 → 30
```

```
0x100C → 40  
0x1010 → 50
```

Cada entero ocupa 4 bytes.

El identificador del arreglo como puntero

El nombre del arreglo representa la dirección del primer elemento.

```
numeros
```

equivale a:

```
&numeros[0]
```

Ejemplo:

```
printf("%p\n", numeros);  
printf("%p\n", &numeros[0]);
```

Ambas líneas imprimen la misma dirección.

Acceso equivalente

Estas expresiones acceden al mismo valor:

```
numeros[2]
```

y

```
*(numeros + 2)
```

Esto ocurre porque un arreglo puede entenderse como un puntero al primer elemento más desplazamiento.

Arreglo bidimensional

Una matriz organiza datos en filas y columnas.

Ejemplo:

```
int tablero[3][3];
```

Carga de matriz

```
for(int i = 0; i < 3; i++) {  
    for(int j = 0; j < 3; j++) {  
        scanf("%d", &tablero[i][j]);  
    }  
}
```

Impresión de matriz

```
for(int i = 0; i < 3; i++) {  
    for(int j = 0; j < 3; j++) {  
        printf("%d ", tablero[i][j]);  
    }  
    printf("\n");  
}
```

Representación en memoria

Aunque se escriba en filas y columnas, en memoria se almacena de forma continua:

```
tablero[0][0]  
tablero[0][1]  
tablero[0][2]  
tablero[1][0]  
tablero[1][1]  
tablero[1][2]
```

Arreglo bidimensional de caracteres

```
char tablero[5][5];
```

Inicialización visual

```
for(int i = 0; i < 5; i++) {  
    for(int j = 0; j < 5; j++) {  
        tablero[i][j] = '~';  
    }  
}
```

```
}  
}
```

Asignación de símbolos

```
tablero[1][2] = 'B';  
tablero[3][4] = 'X';  
tablero[2][1] = 'O';
```

Impresión visual

```
for(int i = 0; i < 5; i++) {  
    for(int j = 0; j < 5; j++) {  
        printf("%c ", tablero[i][j]);  
    }  
    printf("\n");  
}
```

Actividad

Modificar el tablero de batalla naval para que use:

```
char tablero[5][5];
```

Debe permitir:

- inicializar tablero
- colocar barcos
- registrar impacto
- registrar agua
- mostrar tablero actualizado

Extensión opcional: guardar tablero en archivo

```
FILE *archivo;  
archivo = fopen("tablero.txt", "w");
```

Escritura

```
for(int i = 0; i < 5; i++) {  
    for(int j = 0; j < 5; j++) {
```

```
        fprintf(archivo, "%c ", tablero[i][j]);  
    }  
    fprintf(archivo, "\n");  
}
```

Cierre

```
fclose(archivo);
```

Entregar

1. Código actualizado
2. Captura del tablero en ejecución
3. Si realizan extensión, incluir archivo generado